SDP



Module 1: Instructions

⏰ Put the right guidance in the right place

"Copilot is inconsistent when the project context is inconsistent. This module fixes that."

M1 Instructions › M2 Plan Mode › M3 Prompts › M4 Skills › M5 MCP › M6 Agents

⏰ : Monday, 10:30 AM

> "The codebase doesn't tell Copilot what it's for. That's not a Copilot limitation — that's just information nobody wrote down yet."
>
> — David

👨‍💼

David sees missing structure

Every structural question forces Copilot to rediscover the codebase because there is no concise architecture guide.

👩‍💻

Sarah sees repeated review comments

The same standards get re-explained in every PR because the repository has no reliable default guidance.

👩‍🔬

Elena needs context-specific rules

Frontend, backend, tests, docs, and Python utilities each need instructions tuned to them.

Implicit team knowledge becomes explicit working context.

The goal is not more documentation. The goal is better defaults, better prompts, and fewer avoidable corrections.

The Team, Before Module 1

WRONG ANSWERS ONLY

Jessica
The Eager Junior

"Oh, I already did this! I put a comment at the top of every file saying 'be helpful.' Is that not the same thing?"

Confident she solved it six months ago.

David
The Seasoned Architect

"This is a glorified .editorconfig. I've had those since 2009. I'm not sure what the announcement is."

Pedantically wrong. Still technically impressive.

Sarah
The Skeptical Senior

"It reads the entire codebase automatically. You don't have to tell it anything. That's the whole product."

Wrong about the most important part.

Elena
The Quality Champion

"Instructions are suggestions. It'll ignore them the moment something more convenient comes up."

Professionally certain it won't hold.

Marcus
The DevOps Developer

"The instructions file is basically a README. Copilot already reads READMEs, so this is already done. Right?"

Half-right. Wrong half.

Rafael
The Product Visionary

"I just paste our product brief in there and it knows our entire domain. Done in five minutes."

Already scheduling the five-minute meeting.

🎯 What You'll Build

Start broad. Then get specific. Then validate it actually improved.

A layered instruction model that gives Copilot the right context at every level of the codebase

1.0 — A fast starting point

`/init`

Bootstrap a draft instruction baseline from your actual repo — no blank-page starting point.

1.1 — System shape on record

`ARCHITECTURE.md`

Document the system shape so structural questions get faster, clearer answers.

1.2–1.5 — Precision by path

`AGENTS.md + *.instructions.md`

Repo-wide defaults plus targeted rules for frontend, backend, tests, docs, and language runtimes.

1.6 — Domain knowledge encoded

`[show]-universe.md`

A universe file that captures show canon once — so Copilot consults it instead of guessing.

💡 **The instructions don't replace your judgment** — they stop Copilot from making you repeat it.

The Layered Model

Optional enterprise baseline

Organization instructions

Useful when many repositories inherit the same guardrails.

Quick start

`/init` bootstraps a first draft

Let the tool discover patterns first, then refine what it generates.

Context

`ARCHITECTURE.md`

What the system is and where things live.

Baseline rules

`AGENTS.md`

How this repository expects code and content to look.

Precision

Path-based instruction files

Extra rules only when a file pattern actually matches.

This module stops at repository and file-pattern guidance. We keep custom agents for a later module.

🔄 How It Works

The Full Orchestration Sequence

When Copilot evaluates your code, it loads all four sources in parallel, applies a priority hierarchy to resolve conflicts, and generates guidance that's uniquely tailored to your project.

📊 The sequence diagram in detail

We'll walk through the complete orchestration flow showing how all context sources are queried, merged, and prioritized. See the full diagram in the module documentation:

View the orchestration sequence diagram →

What gets loaded
Project context, standards, patterns, and guidance

The priority order
Most specific source wins on conflicts

The outcome
Consistent, team-aligned code suggestions

The benefit
Fewer review comments, faster merges

🔎 Which File Does What?

1 of 5



/init

Copilot's `/init` command — your starting point

Generates a baseline `copilot-instructions.md` or `AGENTS.md` by scanning your repo. Think of it as Copilot reading your codebase and drafting its own briefing document. A good first draft — not a final answer.

WHEN

- Starting a new repo
- Onboarding Copilot to an existing project
- You want a structured starting point

✅ BEST FOR

- Generating a baseline draft
- Capturing existing patterns automatically
- Teams new to instruction files

⚠️ AVOID

- Treating the output as final
- Skipping the review step
- Running it repeatedly without editing

"Wait — it scans the repo and writes its own briefing doc? That's the setup doc nobody ever wrote, written by the thing that needs it. I actually need this."

— Marcus

🔎 Which File Does What?

2 of 5



AGENTS.md

Included with every prompt — automatically, without you asking

This file is silently prepended to every Copilot interaction in the repo. Keep it **short and focused** — conventions, constraints, and defaults only. If it's in here, Copilot always knows it.

✅ PUT THIS HERE

- Coding conventions and naming rules
- Testing framework and style expectations
- Stack defaults (language, libraries, patterns)
- Constraints that apply to every file

⚠️ DON'T PUT THIS HERE

- Rules for specific files or folders
- Architecture or system documentation
- Instructions that only apply sometimes
- Anything too long to stay useful

"Conventions and constraints only. If it 'sometimes applies,' it does not belong here. This file is a contract — treat it like one."

— David

🔎 Which File Does What?

3 of 5



*.instructions.md

Scoped instructions — only active for matching files

When one area of your repo plays by different rules, give it its own instruction file. Uses glob patterns (`applyTo`) to target specific paths — only loaded when those files are in context.

WHEN

- A folder uses a different language
- API vs. UI areas need different rules
- Test files need separate instructions

✅ BEST FOR

- Language-specific rules (Python vs TS)
- Domain-specific patterns per area
- Keeping instructions focused and lean

⚠️ AVOID

- Duplicating repo-wide rules here
- Generic conventions that apply everywhere
- Scoping too broadly (defeats the purpose)

"Loaded only when the relevant files are in scope. That's not a configuration system — that's a context system. The distinction matters quite a lot."

— Elena



ARCHITECTURE.md

The pointer pattern — not official guidance, just good practice

Copilot doesn't automatically read this file. The trick: add a line in `AGENTS.md` that points to it. When that line is in context, Copilot knows to reference the architecture doc — keeping your main instructions file short.

AGENTS.MD

```
# Context
See ARCHITECTURE.md for stack, services, and data flows.
See SECURITY.md for auth patterns and compliance constraints.
See API.md for endpoint conventions and response shapes.
```

✅ WHAT GOES IN ARCHITECTURE.MD

- Tech stack and service overview
- Data flows and system boundaries
- Key architectural decisions and patterns

💡 WHY IT WORKS

- Keeps `AGENTS.md` short
- Add as many pointers as you need
- Each doc stays focused and maintainable
- Works for any context doc, not just architecture



Organization Instructions

Set once at the org level — inherited by every repo

Defined by your GitHub organization, not your repo. Every developer and every repo gets these automatically. Ideal for shared baselines every team inherits instead of re-implementing.

✅ BEST FOR

- Security and compliance guardrails
- Language of acceptable use
- Shared tools and package standards

⚠️ AVOID

- Repo-specific quirks or conventions
- Rules that vary across teams
- Anything that needs frequent updates

⚠️ Important: This is guidance, not compliance

Copilot instructions can always be overridden — by repo-level files, by `*.instructions.md` files, or by the user in chat. Org instructions set a **default baseline**, not a policy wall. Developers retain full control.

💡 Simple Rule

Use the lightest layer that solves the problem. Start broad — specialize only when needed.

🔗 How the Layers Work Together

STEP 1

`/init` discovers existing patterns

THEN ESTABLISH THE SHARED BASELINE

STEP 2A

`ARCHITECTURE.md`

Explains system shape, boundaries, and flows.

STEP 2B

`AGENTS.md` `AGENTS.md`

Defines the repo's default rules.

STEP 3

Add path-based instruction files only where the work changes

Frontend gets frontend guidance. Tests get testing guidance. Generic repo rules do not repeat.

Result: more relevant suggestions, faster setup, fewer avoidable corrections

"I set the context once — architecture here, rules there, exceptions scoped. After that I stopped re-explaining myself on every prompt. Took an afternoon. Paid back by Tuesday."

— Sarah

❌ Before vs ✨ After

❌ Before

Manual instruction writing

Teams spend 60-90 minutes drafting a baseline and still miss conventions hidden across the repo.

Structural questions are slow

Simple prompts force Copilot to inspect the codebase again because nothing explains the system shape clearly.

Generic guidance leaks everywhere

Frontend, backend, tests, docs, and Python utilities all inherit the same vague rules.

✨ After

`/init` gives you a head start

You start with a generated draft, then spend your time refining what matters.

Architecture docs speed up context

Structural questions can move from repo rediscovery toward document-guided answers.

Path-based rules reduce context pollution

The right files receive the right guidance instead of one overloaded rule set trying to do everything.

🔨 Exercise Roadmap

One continuous flow: bootstrap, document the baseline, then add precision where it matters.

#	Exercise	Focus	Lead	Time
1.0	Bootstrap with `/init`	Starting point	Jessica ⭐	8 min
1.1	Create `ARCHITECTURE.md`	System context	David ⭐	10 min
1.2	Create `AGENTS.md`	Repo defaults	Sarah ⭐	10 min
1.3	Path-specific instructions	Context precision	Sarah ⭐	10 min
1.4	Language-specific standards	Language rules	Elena ⭐	8 min
1.5	File-type specialized guidance	Final polish	Marcus ⭐	12 min

The sequence matters.

First establish a baseline. Then document structure. Then add precision where the defaults are too broad.

✅ What Good Looks Like

Architecture guidance

- Tech stack and major folders
- Data and request flow
- Key boundaries and ownership
- No giant file inventory

Repo-wide instructions

- Stable conventions and defaults
- Error handling expectations
- Testing and review preferences
- No file-specific branching logic

Path-based instructions

- Frontend vs backend guidance
- Python vs TypeScript standards
- Tests, docs, Docker specialization
- Small, targeted, and matchable

The goal is an operating model, not just a list of files.

"I was reading Slack. Then I saw 'operating model' and now I'm fully here. That's not a config file — that's a model for how it thinks. Hold on, I need to go update something."

— Marcus

What to Validate in the Workshop

Faster setup

Compare drafting from scratch versus starting with `/init` and refining the result.

Clearer structural answers

Ask a structural question before and after `ARCHITECTURE.md` exists, then compare specificity.

Fewer style corrections

Check whether generated code and tests align more closely with team expectations after the repo and path-based rules are in place.

Measure outcomes you can observe.

This module is stronger when participants can see improved relevance, not just hear abstract best practices.

🔨 Exercises — Instructions and Docs

8 exercises · ~93 minutes · shared leadership

1.0 — Jessica ⭐
/init baseline

Generate a first-draft instruction baseline from your actual codebase

1.1 — David ⭐
Architecture context

Document system shape so structural questions get faster, clearer answers

1.2 — Sarah ⭐
Repo defaults

Set repo-wide coding conventions that apply to every file Copilot touches

1.3 — optional
Layer precision

Apply React patterns to frontend and API rules to backend automatically

1.4 — optional
Language rules

Tailored guidance for Python, JavaScript, and TypeScript in one pass

1.5 — optional
File-type guidance

Specialized rules for tests, Docker configs, and documentation files

1.6 — Sarah ⭐
Universe file

Encode show canon once — Copilot never has to guess the domain again

1.7 — Marcus ⭐
Rebrand & reseed

Use the universe file to rebrand FanHub and populate it with your show's characters, quotes, and lore

Each exercise rotates the lead person — everyone contributes, everyone learns

EXERCISE

1.0 Bootstrap with `/init`

Jessica ⭐ | Sarah 🤝 | 🕐 8 min

Jessica has been hand-copying patterns from senior PR comments into her prompts — one by one, every session.
She is about to find out Copilot can discover the whole repo's conventions automatically.

📋 STEPS

1. Run this in Copilot Chat:

   ```
   /init I'll be using the [language] version of this app.
   ```

2. Review the generated baseline — note which patterns it found, which it missed
3. Consider 2–3 things the AI could not infer on its own — those are yours to add

✅ SUCCESS CRITERIA

- [ ] A baseline file has been generated from the actual codebase
- [ ] You can explain what `/init` discovered automatically
- [ ] You have a list of 2–3 team-specific conventions to add later

💬 *Jessica:* "Wait — it just found all our patterns? I've been copying those from Marcus's PR comments one by one… Does everyone else already know about this?"

1.1

EXERCISE

Create `ARCHITECTURE.md`

David ⭐ | ⏰ 10 min

Copilot keeps rediscovering the same structure because nothing explains the system clearly. **Capture the shape of the project once so structural questions get faster and sharper.**

📋 STEPS

1. Document the tech stack, major folders, flows, and key boundaries
2. Keep it concise - system shape, not every implementation detail
3. Ask a structural question before and after the file exists, then compare the answer

✅ SUCCESS CRITERIA

- ☐ `ARCHITECTURE.md` is created and readable
- ☐ It includes tech stack, folders, flows, and boundaries
- ☐ Subsequent answers become easier to trust

💬 *David: "The real question is: could a new hire read this and model how we think — not just what we do? That's architecture, not documentation."*

📐 Example: ARCHITECTURE.md

📄 THE FILE

```
# FanHub Architecture

## Tech Stack

- Frontend: React 18, Tailwind CSS
- Backend: Express.js, Node.js
- Database: SQLite3
- Deployment: Docker containers

## Folder Structure

fanhub/
├── frontend/ → React components, pages, hooks
├── backend/ → Express routes, middleware, db
├── docs/ → ARCHITECTURE.md, guides
└── infra/ → Docker, compose files

## Key Flows

- User → Frontend → API → Backend routes → DB

## Key Boundaries

- Backend owns all data access
- Frontend owns all UI state
```

💡 What makes this effective

- **Short** — specific and instructive, no sprawl
- **Structural** — stack, folders, flows, boundaries
- **Opinionated** — ownership rules are explicit
- **No noise** — no file inventory, no history

⚠️ WHAT DOES NOT BELONG HERE

- A list of every file and class
- Historical context or changelogs
- Coding conventions (those go in instructions)
- Aspirational architecture not yet built

💡 Elena's note:

Prompt: Creating ARCHITECTURE.md

PROMPT TO COPILOT

Look at this codebase and create a concise `ARCHITECTURE.md` that gives Copilot the structural context it needs to give better suggestions.

This is a navigation aid for Copilot, not a specification. Keep each section focused — omit anything Copilot doesn't need to generate good code.

Cover only:

- Tech stack — languages, frameworks, key libraries
- Folder structure — major directories and what lives in each
- Key flows — how a request moves through the system
- Key boundaries — what owns what, what must not cross
- Patterns — error handling, naming, async conventions

Do not include: a list of every file or class, historical context or changelogs, or architecture that isn't built yet.

Why this prompt works

- **Scoped intent** — "navigation aid, not a spec" focuses Copilot on what it actually needs
- **Named purpose** — "gives Copilot context" focuses it on the right level of detail
- **Explicit sections** — each bullet maps to a section Copilot generates
- **Framing the audience** — "navigation aid, not a spec" prevents over-engineering

AFTER THE FILE EXISTS, VERIFY WITH:

```
Where does a request go after the frontend
calls an API endpoint?
```

David's note:

1.2

EXERCISE

Adjust `AGENTS.md`

Sarah ⭐ | 10 min

The same defaults keep getting re-explained in code review. **Turn repeated feedback into repository-wide guidance that Copilot sees every time.**

📋 STEPS

1. List the conventions that cause the most review comments
2. Turn them into direct, actionable repo-wide instructions
3. Test prompts and check whether the output follows the defaults

✅ SUCCESS CRITERIA

- [ ] Top five review comments are captured
- [ ] Every rule is specific enough to test
- [ ] Generated code needs fewer style corrections

💬 Coach: *"If my team is correcting the same pattern in every PR, that's not feedback — that's a recurring tax. Encode it once. Eliminate it permanently."*

💬 Prompt: Updating AGENTS.md

👇 PROMPT TO COPILOT

We now have ARCHITECTURE.md. Update AGENTS.md to reflect that.

Specifically:

- Remove any detailed tech stack descriptions — those belong in ARCHITECTURE.md (frameworks, versions, request flow diagrams)
- Replace them with a single reference: `See ARCHITECTURE.md for system structure and key boundaries`
- Keep all coding standards, conventions, and behavioral rules in AGENTS.md — those are the agent's instructions
- Don't duplicate information that now lives in ARCHITECTURE.md

The goal: instructions that rarely change, and architecture context that stays in the right file.

💡 Why this matters

- **Single source of truth** — stack details live in ARCHITECTURE.md, not duplicated in two files
- **Lower maintenance cost** — when the stack changes, update one file
- **Cleaner signal** — instructions stay focused on behavior, not facts

BEFORE → AFTER

~~Frontend: React 18, Tailwind CSS~~
~~Backend: Express.js, Node.js~~
~~Database: SQLite3~~

```
See ARCHITECTURE.md for system
structure and key boundaries.
```

💡 Sarah's rule:

⚠️ Critical Scope Boundary

AGENTS.md is for Conventions & Constraints, NOT Details

✅ Put Here

- How we write code (async/await, not promises)
- What we avoid (no class components)
- Team standards (Jest for testing, Tailwind for CSS)
- Error handling expectations (always catch errors)

❌ Put Elsewhere

- Architecture diagrams (belongs in ARCHITECTURE.md)
- API documentation (use docs/ or separate files)
- Detailed how-to guides (path-based files)
- Full system context (too much for a baseline)

💡 The Rule of Thumb

If it's something Copilot needs to remember on every single prompt, it belongs in AGENTS.md. If it's something developers need to read once, it belongs elsewhere.

1.3

EXERCISE

Path-Specific Instructions

Sarah ⭐ | 10 min

Frontend and backend keep borrowing the wrong patterns from each other. **Use path-specific instruction files so each area gets the guidance it actually needs.**

STEPS

1. Create `.github/instructions/`
2. Add focused frontend and backend instruction files
3. Verify each file activates only where intended

SUCCESS CRITERIA

- [ ] Matching files receive the right layer guidance
- [ ] Cross-layer suggestion noise decreases
- [ ] Repo-wide defaults are not duplicated

💬 *Sarah: "Precision beats volume. The right rule in the right place wins."*

💬 Prompt: Path-Specific Instructions (E1.3)

🤔 PROMPT TO COPILOT

Create two path-scoped instruction files in `.github/instructions/`.

File 1: `frontend.instructions.md`

- `applyTo: "frontend/**"`
- React patterns: functional components, hooks, no class components
- State management: when to use local state vs context
- Performance: memoization, avoiding unnecessary re-renders
- Styling: Tailwind only, no inline styles

File 2: `backend.instructions.md`

- `applyTo: "backend/**"`
- Error handling: always use try/catch, never swallow errors silently
- Validation: validate all request bodies before processing
- Logging: use structured logging, include request IDs
- Security: never log sensitive data (passwords, tokens, PII)

Do not repeat anything already in `AGENTS.md`.

💡 Why this works

- **applyTo scoping** — guidance activates only for matching files, not everywhere
- **No duplication rule** — keeps repo-wide defaults in one place
- **Two files, two audiences** — frontend and backend conventions no longer compete

VERIFY WITH

```
Open a frontend file and ask Copilot to add
a
new component. Check it follows React
patterns.
Open a backend route. Ask for error
handling.
Confirm it doesn't suggest React patterns.
```

💡 Sarah's rule:

1.4

EXERCISE

Language-Specific Standards

Elena ⭐ | 8 min

Python, JavaScript, and TypeScript each have different expectations. **Use file-extension patterns so each language gets guidance that feels native.**

STEPS

1. Create separate files for Python, JavaScript, and TypeScript
2. Encode the language rules that matter most for your team
3. Confirm each pattern activates on the right extensions

SUCCESS CRITERIA

- [ ] Language-specific files are targeted correctly
- [ ] Cross-language contamination drops
- [ ] Suggestions feel native to each language

💬 *Elena: "Good guidance makes the right call feel obvious. But what about when 'obvious' conflicts across contexts? That's the edge case your instruction file is quietly ignoring."*

Prompt: Language-Specific Instructions (E1.4)

PROMPT TO COPILOT

Create three language-specific instruction files in `.github/instructions/`.

python.instructions.md — `applyTo: "**/*.py"`

- PEP 8 naming: snake_case functions, Capitalized
- Always include type hints on parameters and
- Triple-quoted docstrings with

javascript.instructions.md — `applyTo: "**/*.js"`

- ES modules, no
- Prefer

typescript.instructions.md — `applyTo: "**/*.ts,**/*.tsx"`

- Strict mode, no
- Use

Why this works

- **Glob by extension** — `**/*.py` activates only for Python files across any folder
- **Language idioms, not project rules** — these travel well across repos
- **Three files, three contexts** — no cross-language noise in suggestions

VERIFY WITH

```
Open a .py file and ask Copilot to add a
helper
function. Check: type hints present,
snake_case,
docstring included.
```

Elena's note:

1.5

EXERCISE

File-Type Specialized Guidance

Marcus ⭐ | ⏰ 12 min

Tests, Docker files, and docs serve different purposes than production app code. **Create file-type rules that optimize for clarity, safety, and audience.**

📋 STEPS

1. Add targeted files for tests, Docker, and documentation
2. Use precise `applyTo` patterns for each purpose
3. Check that production-code advice no longer leaks into those files

✅ SUCCESS CRITERIA

- ☐ Test files optimize for clarity and edge cases
- ☐ Docker files emphasize security practices
- ☐ Docs receive guidance suited to their audience

💬 *Marcus: "Oh — here's the thing. The best instruction file isn't one you read. It's one you never have to. Like good Wi-Fi — you only notice when it's gone. ...anyway."*

Prompt: File-Type Instructions (E1.5)

PROMPT TO COPILOT

Create three file-type instruction files in `.github/instructions/`.

tests.instructions.md

- `applyTo: "**/__tests__/**,**/*.test.{ts,tsx,js}"`
- Test both happy path and error cases
- Mock external API calls and database queries
- Test behavior, not implementation details

docker.instructions.md

- `applyTo: "**/Dockerfile*,**/docker-compose*"`
- Must change instructions, base image, filesystem, save images

docs.instructions.md

- `applyTo: "docs/**,**/*.md"`
- Plain language, no jargon, examples over descriptions

Why this works

- **Purpose-specific rules** — tests, infra, and docs each have a different job; one set of rules can't cover all three
- **Glob by purpose** — test files match by pattern, Docker by filename prefix
- **Narrow and specific** — each file stays under 15 lines

VERIFY WITH

```
Open a test file and ask Copilot to add a test
case for a failing API call. Confirm it mocks
the external call and tests the error branch.
```

Marcus's take: "I'd never thought about giving Docker files their own instructions. But yeah — multi-stage builds, and the only safe approach and I was tired of explaining that."

1.6

EXERCISE

Define Your Universe

Sarah ⭐ | David 🤝 | ⏱ 15 min

This is where you make FanHub your own — swap Breaking Bad for any show your team loves. Encode that show's canon once so Copilot never has to be told again.

📋 STEPS

1. **Ground the content** — any TV series, game, or universe you know well. Then in Agent mode, scaffold `docs/[show]-universe.md` using web search for premise, characters, terminology, and canon rules
2. Review and correct the draft — find at least one inaccuracy Copilot introduced
3. Add a `## Universe` pointer section to `AGENTS.md` linking to the file
4. Validate: ask Copilot for a character bio — expect accurate, show-specific content on the first pass

✅ SUCCESS CRITERIA

- ☐ `[show]-universe.md` drafted in Agent mode
- ☐ At least one Copilot error found and corrected
- ☐ 5+ characters, 3+ terms, 1+ canon accuracy rules
- ☐ `AGENTS.md` links to universe file
- ☐ Bio is [illegible] accurate, show-specific [illegible]

> 💬 *Sarah:* [illegible]

Prompt: Creating the Universe File (E1.6)

✨ **This is where you make FanHub yours.** Replace `Breaking Bad` with any show your team loves — the rest of the workshop uses your universe file.

PROMPT TO COPILOT (AGENT MODE)

```
Use #file:breaking-bad-universe.md as the exemplar and let's create a file to contain domain knowledge for my favorite show. Use web search to research [your show] and create docs/[show]-universe.md.

Cover these sections:
• Premise — one paragraph summary of the show
• Main characters — name, role, key traits, status (alive/deceased)
• Key locations — recurring places and their significance
• Terminology — show-specific terms and lore the team uses
• Canon accuracy rules — common mistakes to avoid
```

💡 Why Agent mode + web search

- **Web search** — fetches current, accurate canon instead of relying on training data alone
- **Agent mode** — writes the file directly; no copy-paste needed
- **Confidence levels** — forces Copilot to surface uncertainty so you know what to verify

⚠️ ALWAYS REVIEW THE OUTPUT

- Check character statuses — alive/deceased is commonly wrong
- Verify titles and roles — Copilot often upgrades them

The exercise goal: find at least one error before saving the file.

💡 The payoff:

1.7

EXERCISE

Rebrand and Reseed FanHub

Marcus ⭐ | Rafael 🤝 | ⏰ 20 min

The universe file exists. Now use it. Ask Copilot to rebrand FanHub for your show — update the UI copy, app title, and any Breaking Bad references. Then use Agent mode to generate real seed data: characters, quotes, and lore, drawn directly from the file you just created.

📋 STEPS

1. In Agent mode, ask Copilot to rebrand FanHub for your show — update the app title, nav copy, and any Breaking Bad references in the UI
2. Ask Copilot to generate seed data using `docs/[show]-universe.md` as context — at least 3 characters and 5 quotes per main character
3. Apply the seed data and run the app — verify your show's content appears throughout
4. Ask Copilot a domain question about your show without any extra context — does it answer from the universe file?

✅ SUCCESS CRITERIA

- ☐ App title and UI copy match your chosen show
- ☐ No Breaking Bad references remain in the visible UI
- ☐ Seed data includes 3+ characters and 5+ quotes from your show
- ☐ Copilot answers a domain question about your show without being told to

💬 *Marcus:*

Prompt: Rebrand + Reseed (E1.7)

✨ **The universe file does the explaining.** You wrote it once — now Copilot reads it for every task that follows.

PROMPT TO COPILOT (AGENT MODE)

```
FanHub is currently branded for Breaking Bad.
Using docs/[show]-universe.md as context,
rebrand it for [your show] and regenerate its
seed data.

Rebrand:
• Update the app title, nav copy, and any
Breaking Bad references in the UI
• Update the color scheme and styles to match
the tone of your show
• Do not change app structure, routes, or data
schema

Reseed:
• Find the existing seed data and replace it
with content from the universe file
```

💡 Why this works

- **Universe file = zero re-explaining** — Copilot reads your canon before generating anything
- **Language-agnostic** — Copilot finds the right files from context; same prompt works across all implementations
- **"Flag uncertain quotes"** — forces Copilot to surface what it's guessing, so you only review what needs it

⚠️ ALWAYS VERIFY QUOTES

- Copilot paraphrases — check verbatim accuracy on anything the app will display
- Character bios are usually reliable; episode-specific facts less so
- If in doubt, the universe file you wrote is the source of truth

💡 **The payoff:**

✅ Module Checklist

Quick Start

- Run `/init`
- Review what it generated
- Note what needs refinement

Foundation

- Create `docs/ARCHITECTURE.md`
- Create `AGENTS.md`
- Confirm both describe reality clearly

Precision

- Add matching instruction files
- Verify file patterns behave as intended
- Avoid duplicating baseline repo rules

Domain Knowledge

- Create `docs/[show]-universe.md`
- Add `## Universe` pointer to instructions
- Validate with a character bio test

You now have a launchpad.

Copilot knows your repo, your conventions, and where to look. Everything from here builds on this.



Module 1 Is Locked In

🧠 Next Up: Module 2 — Agent Plan Mode

You Built the Baseline

`/init` architecture context, and repo defaults are in place

You Added Precision

Path-, language-, and file-specific guidance shapes behavior where it matters

Now Plan Before Coding

Module 2 teaches Copilot to think through the work before it starts typing

Better instructions create better starting conditions. **Module 2 turns that strong context into better decisions.**

Reflect & Discuss

STORY MOMENTS

The PR patterns Jessica had been manually copying from Marcus were already living in old conversations — Copilot found them instantly

David realized the codebase doesn't tell Copilot what it's *for* — architecture and intent have to be written down explicitly

Sarah's 12 recurring weekly questions could disappear into a single file — answered once, referenced forever

ONE BIG QUESTION

What's one piece of context about your team or codebase that Copilot couldn't possibly know right now — and what would change if it did?

The Team, After Module 1

✓ AFTER THE MODULE

Jessica

The Eager Junior

"It turns out 'be helpful' was not a context file. It just found all our patterns — I've been copying those from Marcus's PR comments one by one. Does everyone else already know about this?"

The moment imposter syndrome took a lunch break. Also: Marcus knows.

David

The Thoughtful Architect

"The codebase doesn't tell Copilot what it's for. An .editorconfig doesn't either — and I've had those since 2009. Neither is the point."

Twenty years of hard lessons. Turns out one was missing.

Sarah

The Skeptical Senior

"It does not read the codebase automatically. I know this because I've answered 'What is Breaking Bad about?' twelve times this week. After this, the answer lives in the file. Zero times next week."

Wrong about the auto-reading. Right to be annoyed about the twelve times.

Elena

The Pragmatic Engineer

"It held the instruction. Fine. But does your TypeScript rule activate on a .tsx file? That's the one that bites. I'm still checking."

Instructions held. She moved the goalposts. Professionally.

Marcus

The Curious Tinkerer

"It's not a README. It's the Wi-Fi router. You don't read it — you just stop noticing the friction. ...anyway."

Clicked when he found a better analogy than his own.

Rafael

The Pragmatic Lead

"I did paste the product brief. It helped. Then I found out the architecture context was wrong and it had been guessing for weeks. One file fixes both. Now I'm thinking about Module 2."

Partially right. Immediately planning the next fix.